

I used lists and sets as the main data structures for this product recommendation program. The product catalog is stored as a list of dictionaries because each product has multiple pieces of information, including its name and tags. Lists make it easy to loop through every product in the catalog. Each product's tags are converted from a list to a set, and the customer's preferences are converted to a set as well.

The most important operation in the program is the set intersection. The `count_matches()` function finds the tags a product and a customer have in common using the `intersection()` method. The length of the resulting set gives the number of matching preferences. Sets are useful here because they are designed for membership and comparison operations and automatically eliminate duplicate values.

I also used loops to collect customer preferences and to check every product in the catalog. The recommendation function stores products that match at least one preference and sorts them by their match score, so that products with the strongest matches appear first.

If the catalog contained 1,000 or more products, this basic approach could still work, but the program would need to be designed more carefully for performance. For a larger system, I could preprocess product tags, store frequently used information in a database, and avoid repeatedly processing the same data. A production recommendation system could also consider additional information such as purchase history, product popularity, price, ratings, and customer behavior. However, sets and simple iteration are appropriate for this small prototype because they keep the program easy to understand while demonstrating efficient tag comparisons.